CSC 367 – Assignment 2

PART 1.

Answer the following questions directly on this paper and hand in your answers. No electronic submissions are accepted. Your pages should be stapled together, with identification (student number at least). A neat and professional appearance is expected and marks will be deducted otherwise.

Question 1. [5] The Motorola 68000 microprocessor is byte addressable with a 24-bit address bus and a 16-bit data bus, and one word contains 2 bytes. Your manager wants to build a system using 32 peripherals each of which is byte addressable and requires an address space of 128KB. A byte addressable memory unit of 8MB has already been bought and the manager asks you to check that it is all feasible.

(a) Is it? YES NO (circle one)

(b) Show a complete logical explanation of the reasoning for your answer, as you might do at work.

(c) Following up from above, if you answered that 32 peripherals are not possible, then state what is the maximum number that could be attached. Conversely, if you answered that the 32 peripherals are possible, then state how many more units could be attached, if any. In either case, justify your answer.

Question 2. [24] Fill in the table below with the appropriate information about the ARM instructions.

Instruction	Explanations
`MOV R5,#0x42`	*Addressing Modes of ALL the Operands* Register, Immediate Value
	What this instruction does (in precise english) and what the results are, if it is possible to state them with certainty Sets Register 5 to a value of 66

`LDR R5,=Test`
where
`Test: .skip 4`
	Addressing Modes of ALL the Operands
	Register, Relative address

	What this instruction does (in precise english) and what the results are, if it is possible to state them with certainty Sets Register 5 to the address of the Test variable

`STR R4,[R5]`
where at this moment
`r5=0xC003;`
`r4=0xC004;`
	Addressing Modes of ALL the Operands
	Register, Zero Offset

	What this instruction does (in precise english) and what the results are, if it is possible to state them with certainty Stores the value 0xC004 to memory address 0xC003

`STR R5,[R4,#4]!`
where at this moment
`r5=0xC003;`
`r4=0xC004;`
	Addressing Modes of ALL the Operands
	Register, Pre-indexed Immediate Offset With Writeback

	What this instruction does (in precise english) and what the results are, if it is possible to state them with certainty Stores the value 0xC003 to memory address 0xC008, then sets register 4 to value 0xC008

Instruction	Explanations
LDR R1,[R2],#8 ***where at this moment*** **r1=0xC003;** **r2=0xC004;**	*Addressing Modes of ALL the Operands* Register, Post-Indexed immediate offset *What this instruction does (in precise english) and what the results are, if it is possible to state them with certainty* Register 1 is set to the value at memory address 0xC004, then Register 2 is set to 0xC008
ADD R1,R2,R3 ***where at this moment*** **r1=0xC016; r2=0x0032;** **r3=0x0032;**	*Addressing Modes of ALL the Operands* Register, Register, Register *What this instruction does (in precise english) and what the results are, if it is possible to state them with certainty* Register 1 is set to 0x0032 plus 0x0032
ADDS R1,R2,R3,ASL #4 ***where at this moment*** **r1=0xAC16; r2=0xAB32;** **r3=0x0004;**	*Addressing Modes of ALL the Operands* Register, Register, Arithmetic shift left immediate *What this instruction does (in precise english) and what the results are, if it is possible to state them with certainty* R1 = 0xAB32 + (0x0004 x 2^4)
LDR R1,[R2,R3] ***where at this moment*** **r1=0xC013; r2=0xC008;** **r3=0x0004;**	*Addressing Modes of ALL the Operands* Register, Pre-indexed register offset *What this instruction does (in precise english) and what the results are, if it is possible to state them with certainty* R1 = the contents of the mem address at 0xC00C

Instruction	Explanations
`ORR R1,R2,R3` ***`where at this moment`*** **`r1=0xC013; r2=0xC001; r3=0xC028;`**	*Addressing Modes of ALL the Operands* Register, Register, Register
	What this instruction does (in precise english) and what the results are, if it is possible to state them with certainty R1 = binary or of 0xC001 and 0xC028 R1 = 0xC029
`SUB R1,[R2]` ***`where at this moment`*** **`r1=0xC013; r2=0xC008;`**	*Addressing Modes of ALL the Operands* Register, Zero Offset
	What this instruction does (in precise english) and what the results are, if it is possible to state them with certainty Nothing, invalid syntax
`LDR R1,[R2,R3]!` ***`where at this moment`*** **`r1=0xC013; r2=0xC008; r3=0x0004;`**	*Addressing Modes of ALL the Operands* Register, Pre-indexed Register Offset With Writeback
	What this instruction does (in precise english) and what the results are, if it is possible to state them with certainty R1 = contents of mem address 0xC00C. R2 = 0xC00C
`MOV R2,[R3]` ***`where at this moment`*** **`r2=0xC008; r3=0x0004;`**	*Addressing Modes of ALL the Operands* Register, Zero Offset
	What this instruction does (in precise english) and what the results are, if it is possible to state them with certainty Nothing, invalid syntax Stores the value 0xC003 to memory address 0xC008, then sets register 4 to value 0xC008

Question 3. [10] In many programming languages there exist 3 different types of control structure for iteration: (1) a *REPEAT* structure (often called "DO...WHILE"); (2) a *WHILE* structure; (3) a *FOR* structure. As a reminder, the corresponding C or Java code for WHILE and REPEAT is given below.

```
REPEAT:
   do {
        [perform actions]
   } while (condition)
```

```
WHILE:
   while (condition) {
        [perform actions]
   }
```

The following *pseudocode* segment uses a WHILE structure.

```
assign Count the value x;
while (Count < y)
{
     [print the value assigned to Count and increase Count by 1]
}    end while loop
```

The following *pseudocode* segment uses a REPEAT structure and maintains the logical integrity of the previous segment.

```
assign Count the value x;
if (Count < y)
{
  do
  [print the value assigned to Count and increase Count by 1]
  while (Count <y)
}    end repeat loop
```

The difference between the two looping structures is that, in general, a REPEAT one always executes through the body of the loop at least once before any condition is tested. In the WHILE loop instead the condition is tested before even entering the loop and thus the body of the loop may never be executed. The second pseudo code reflects these paradigms and makes sure that there is an extra testing of the condition before doing a REPEAT loop in order to preserve the expected behaviour previously given by the WHILE loop implementation.

Using the pseudocode above a basis, complete the ARM code in Figure 1. You must write two complete segments of code to implement both the pseudocode for the WHILE loop and the pseudocode for the REPEAT loop. Each ARM code must reflect the structure of the pseudocode as precisely as possible. You code should assume the existence of a call to some fake "`print_int`" function to print an integer, where the value to print is in register R0, as shown in lines [5] and [6].

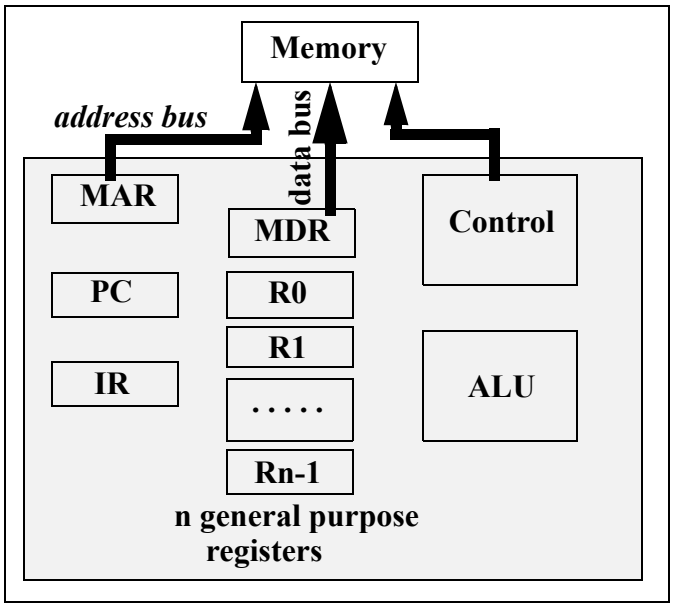
Your answer must include 4 parts:

(a) **[1]** A small flowchart reflecting the structure of the WHILE loop as it will be implemented in assembly language. This is not necessarily the flowchart directly from the pseudocode above, as explained in the lectures, labs and notes.
(b) **[4]** The ARM code for the WHILE loop using the segment of Figure 1 as a start.
(c) **[1]** A small flowchart reflecting the structure of the REPEAT loop as it will be implemented in assembly language. This is not necessarily the flowchart directly from the pseudocode above, as explained in the lectures, labs and notes.
(d) **[4]** The ARM code for the REPEAT loop using the segment of Figure 1 as a start.

```
[1]  LDR  R1,=X     @ R1 = address of X
[2]  LDR  R1,[R1]   @ R1 = Count = value of X
[3]  LDR  R2,=Y     @ R2 = address of Y
[4]  LDR  R2,[R2]   @ R2 = value of Y
     . . . . . .
to print, use:
     MOV  R0,R1     @ R0 = X (to be printed)
     BL   print_int @ call function to print an integer
```

Figure 1: The initial ARM code

Question 4. [5] *(This question was given in the Midterm 1 in Spring 2008).* **C**onsider the figure below, from the lecture notes, showing the main components of a possible CPU and its interface to memory. Then analyze the general **CISC** instruction: `ADD R3,8(R4)`, whose semantics are: "*Assign to R3 the result of adding the current value in R3 to the value from a location in memory, whose effective address is computed as: the sum of the current content of R4 plus the constant preceding it*". Describe the precise steps which occur in the Fetch / Decode / Execute cycle. State which registers, buses, control lines, etc. are used. Be brief yet precise. Use pseudo-code or similar, do not write an essay!



ADD R3,8(R4) ***STEPS FOR FETCH CYCLE:***

1. PC = Mem Address
2. MAR <- PC
3. MAR puts address on bus
4. CPU reads mem with control bus signal
5. 32-bit instruction is returned on bus to MDR
6. IR <- MDR
7. PC += 4

ADD R3,8(R4) ***STEPS FOR DECODE AND EXECUTE CYCLE:***

Decode:
1. Decoder recognizes ADD instruction

Execute:
1. ALU computes addition of R3 and 8
2. Output of ALU stored in R3

Question 5. [16] Examine carefully the ARM mystery code below and step through it, manually executing the loop 7 times to find out what it is supposed to do. The numbers on the left denote line numbers for easy reference. At the end you should be able to state what the code is supposed to do, what are the final values at the end of the execution of the 7 loops, whether there is something incorrect and, if so, how to fix it. Answer the detailed questions.

(a) [2] Show the contents of the two arrays `A1` and `W1` after executing the loop 7 times. Write each element in one slot below.

A1 = 3 1 8 1 4 5 1 -1

W1 = 1 3 6 ___ ___ ___ ___ ___

(b) [6] Show the final contents of the registers after the loop is executed 7 times.

R1 = 3	**R2** = 6	**R3** = 1
R4 = 1	**R5** = =W1	**R6** = =A1 + 21

```
            .text
            .global_start
[1]   _start:    LDR   R6,=A1
[2]              LDR   R5,=W1
[3]              MOV   R1,#0
[4]              MOV   R2,#0
[5]              MOV   R4,#1

[6]   LOOP:      LDR   R3,[R6],#4          @loads the next num in A1
[7]              CMP   R4,R3               @checks if num == 1
[8]              BNE   INCR                @if so, go to incr method

[9]              STR   R2,[R5,R1,ASL #2]   @stores r2 to w1[r1]
[10]             ADD   R1,R1,#1            @increments r1

[11]  INCR:      ADD   R2,R2,#1            @Increments r2
[12]             BAL   LOOP                @loop
[13]             SWI   0x11
            .data
            .align
      A1:   .word     3,1,8,1,4,5,1,-1
      W1:   .word     0,0,0,0,0,0,0
            .end
```

(c) [2] Write appropriate and meaningful comments *next to the code above*, showing you understand not just what the local instruction is doing, but also what the actual effect is supposed to be. For example, `MOV R1,#1` should not be commented as in: *"R1=1"*, but perhaps it would be good to state: *"Set R1 as initial loop counter to 1"*.

(d) [2] What do you think the code is trying to do?

Store the indexes where the 1s are

(e) [2] Something is incorrect in the code. Can you state what it is?

There is no way to exit

(f) [2] Can you give one or more instructions to fix the problem and state exactly where the instructions should be placed?

change INCR

```
ADD R2, R2, #1
CMP R2, #7
BLT LOOP
```

Question 6. [10] Assume the following computations are executed *sequentially*. For each, give the corresponding ARM instruction(s) and the contents of register R5 after execution. Feel free to test the correctness of your answer using the simulator.

Computation	ARM instruction(s)	Content of R5 in Hex
Place the Hexadecimal constant CB2F 6F2A$_{16}$ in R5	MOV R5, #0xCB2F6F2A	0xCB2F6F2A
OR the second byte from the right (byte 1) of R5 with F0$_{16}$ (only that byte)	ORR R5, R5, #0xF000	0xCB2FFF2A
R1 = 2$_{16}$ and you want to AND it with R5, with the result in R5	AND R5, R5, R1	0000002
Complement every bit in R5	MVN R5, R5	0xFFFFFFFD
EX-OR the leftmost byte of R5 with 5F$_{16}$	EOR R5, R5, #0x5F000000	0x A0FFFFFD

Question 7. [10] Everybody is keenly aware that we must learn to consume less energy. A lot is talked about fuel consumption for driving, using a clothes dryer, or not wasting high power lights. Not many people think about the power consumption of computer systems, whether relating to the "Google server farms" or the decision to leave their own machine on, perhaps in sleep mode, all night. Your task is to find precise enough data to raise everybody's awareness of the energy (electrical) impact of computing systems at least in North America. You might only be able to find data for the US and not separately for Canada. Choose to follow through in details researching at least *two* of the questions below. If you answer well more than one, extra marks are also available. It is crucial that you include a reference to the source of your data. Feel free to attach, even electronically, a copy of articles you find, as the best answers and sources will be posted.

(a) Google supports a large amount of servers in many clusters located around the world. How much electricity is used to support them?

(b) Find some data about the estimated percentage of electricity used in North America for computer systems compared to the total (there have been articles).

(c) If a laptop is in sleep mode, or a monitor is turned off in a desktop machine, is there any power still being used? What about a printer, a scanner, a router? Analyze the typical setup of a fairly technical home with at least 2 people, a modem/cable connection with a router, one desktop and one laptop, a shared printer and a shared scanner. How much is being consumed in electricity? Is it important to turn everything off when not in use? Should everything actually be unplugged as well? What does hibernation or sleep mode give exactly?

(d) Assuming similar functionality and features among laptops, is it possible to discriminate among them for their energy use? This is not the same as noting how long their battery lasts.

(e) Is it possible to discriminate among LCD monitors of the same size for their energy use?

(f) List at least 5 processors found in desktop machines, Windows, Mac or Unix. Find their energy consumption.

Attach a separate paper for your answer.

Question 8. [20] Given the figure below showing the main components of a possible CPU, its internal data-path and its interface to memory through the Address bus/MAR and Data bus/MDR connections, analyze the ARM instructions listed below. Describe the steps which occur when each instruction is fetched and executed. Give precise details of everything that happens, which registers, buses, control lines, etc. are used, how each element is addressed and describe the purpose of each micro-operation. Be brief yet precise. Follow the scheme and the answers shown in the lectures for the three phases (Fetch, Decode, Execute) and the amount of details given there as an example.

The 5 ARM Instructions to be analyzed: (Attach an extra sheet for your answer)

```
[1]   MOV   R1,#0x44
[2]   LDR   R2,[R3,#4]
[3]   MUL   R4,R1,R2
[4]   ADD   R4,R1,R2,LSL #2
[5]   STR   R4,[R3],#4
```

